

```

# Initialize a 3x3 board using a 2D list.

# We use a space ' ' to represent an empty cell.
board = [[' ' for _ in range(3)] for _ in range(3)]

def print_board():
    """Prints the current state of the board."""
    print()
    for i in range(3):
        # Print the current row with vertical line separators
        print(board[i][0] + " | " + board[i][1] + " | " + board[i][2])

        # Print a horizontal line after the first and second rows
        if i < 2:
            print("--+---+--")
    print()

def check_winner(player):
    """
    Checks if the specified player ('X' or 'O') has won the game.
    Returns True if a win condition is met, otherwise False.
    """
    # 1. Check all 3 Rows for a match
    for i in range(3):
        if all(board[i][j] == player for j in range(3)):
            return True

    # 2. Check all 3 Columns for a match
    for j in range(3):

```

```
    if all(board[i][j] == player for i in range(3)):
        return True
```

3. Check the Main Diagonal (Top-Left to Bottom-Right)

```
if all(board[i][i] == player for i in range(3)):
    return True
```

4. Check the Anti-Diagonal (Top-Right to Bottom-Left)

```
if all(board[i][2 - i] == player for i in range(3)):
    return True
```

If no matches are found, the player hasn't won yet

```
return False
```

```
def is_full():
```

```
    """Checks if the board is completely full (resulting in a draw)."""
```

```
    for row in board:
```

```
        if ' ' in row:
```

```
            return False # Found an empty space, so it's not full
```

```
    return True # No empty spaces left
```

```
def player_move():
```

```
    """Handles the human player's ('X') turn."""
```

```
    while True:
```

```
        try:
```

```
            # Get row and column input from the user
```

```
            row = int(input("Enter row (0-2): "))
```

```
            col = int(input("Enter column (0-2): "))
```

```

# Validate the move: check if it's within bounds AND the cell is empty
if 0 <= row < 3 and 0 <= col < 3 and board[row][col] == ' ':
    board[row][col] = 'X' # Place the 'X' on the board
    break # Exit the loop since a valid move was made
else:
    print("Invalid move! The cell is either taken or out of bounds. Try again.")
except ValueError:
    print("Invalid input! Please enter a number between 0 and 2.")

```

```
def computer_move():
```

```
    """
```

```
    Handles the computer's ('O') turn using a simple Brute Force strategy.
```

```
    It evaluates moves in a specific order of priority.
```

```
    """
```

```
    print("Computer's turn...")
```

```
    # Strategy 1: Try to Win
```

```
    # Loop through all cells to see if placing an 'O' results in an immediate win
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if board[i][j] == ' ':    # If cell is empty
```

```
                board[i][j] = 'O'    # Temporarily place 'O'
```

```
                if check_winner('O'): # Does this move win the game?
```

```
                    return            # Yes! Keep the 'O' there and end the turn
```

```
                board[i][j] = ' '    # No win, so undo the temporary move
```

```
    # Strategy 2: Block the Player
```

```
    # Loop through all cells to see if the player ('X') is about to win on their next turn
```

```
    for i in range(3):
```

```

for j in range(3):
    if board[i][j] == ' ':    # If cell is empty
        board[i][j] = 'X'    # Temporarily place an 'X'
        if check_winner('X'): # Would this move make the player win?
            board[i][j] = 'O' # Block them by placing our 'O' here instead!
        return                # End the turn
    board[i][j] = ' '        # Undo the temporary move

```

Strategy 3: Pick the first available empty space

If the computer can't win and doesn't need to block, just pick the first empty spot

```

for i in range(3):
    for j in range(3):
        if board[i][j] == ' ':
            board[i][j] = 'O'
            return # End the turn

```

```

def play_game():

```

```

    """Main game loop that alternates turns and checks for game over conditions."""

```

```

    print("--- Tic-Tac-Toe Game ---")

```

```

    print_board()

```

```

    while True:

```

```

        # Human Player's Turn

```

```

        player_move()

```

```

        print_board()

```

```

        if check_winner('X'):

```

```

            print("You win!")

```

```

            break

```

```

        if is_full():

```

```

        print("It's a draw!")
        break

# Computer's Turn
computer_move()
print_board()
if check_winner('O'):
    print("Computer wins!")
    break
if is_full():
    print("It's a draw!")
    break

```

Call the play_game function to start the program

play_game()

Here are all the remaining 13 lab programs from your PDF, fully corrected (fixing the syntax errors, missing colons, and typo artifacts from the PDF) and heavily commented block-by-block so you can easily understand the logic.

AI Termwork 2: Tic-Tac-Toe using Minimax Strategy

Python

Initialize a 1D board with 9 empty spaces

```
board = [' ' for _ in range(9)]
```

```
def print_board():
```

```
    """Prints the 3x3 board."""
```

```
    print("\n")
```

```
    print(board[0] + " | " + board[1] + " | " + board[2])
```

```
    print("--+---+--")
```

```
    print(board[3] + " | " + board[4] + " | " + board[5])
```

```

print("--+---+--")
print(board[6] + " | " + board[7] + " | " + board[8])
print("\n")

```

```

def check_winner(player):
    """Checks all 8 possible winning combinations for the given player."""
    win_positions = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8], # Rows
        [0, 3, 6], [1, 4, 7], [2, 5, 8], # Columns
        [0, 4, 8], [2, 4, 6]           # Diagonals
    ]
    for pos in win_positions:
        # Check if all three indices in a win position match the player's symbol
        if board[pos[0]] == board[pos[1]] == board[pos[2]] == player:
            return True
    return False

```

```

def is_full():
    """Checks if there are no empty spaces left."""
    return ' ' not in board

```

```

def minimax(is_maximizing):
    """
    Minimax algorithm: Recursively evaluates all possible moves.
    Returns 1 if 'O' wins, -1 if 'X' wins, and 0 for a draw.
    """
    # Terminal states (Base cases)
    if check_winner('O'): return 1
    if check_winner('X'): return -1

```

```
if is_full(): return 0
```

```
if is_maximizing:
```

```
    # Computer's turn ('O') -> Wants the highest score
```

```
    best_score = float('-inf') # Initialize to lowest possible
```

```
    for i in range(9):
```

```
        if board[i] == ' ':
```

```
            board[i] = 'O'
```

```
            score = minimax(False) # Call minimax for the opponent's turn
```

```
            board[i] = ' ' # Undo move
```

```
            best_score = max(score, best_score)
```

```
    return best_score
```

```
else:
```

```
    # Human's turn ('X') -> Wants the lowest score
```

```
    best_score = float('inf') # Initialize to highest possible
```

```
    for i in range(9):
```

```
        if board[i] == ' ':
```

```
            board[i] = 'X'
```

```
            score = minimax(True)
```

```
            board[i] = ' '
```

```
            best_score = min(score, best_score)
```

```
    return best_score
```

```
def computer_move():
```

```
    """Finds the best move for the computer using Minimax."""
```

```
    best_score = float('-inf')
```

```
    move = -1
```

```
    for i in range(9):
```

```
        if board[i] == ' ':
```

```
board[i] = 'O'

score = minimax(False) # Evaluate this move

board[i] = ' '

if score > best_score:

    best_score = score

    move = i

board[move] = 'O' # Make the best move
```

```
def player_move():

    """Handles human input."""

    while True:

        try:

            move = int(input("Enter position (1-9): ")) - 1

            if 0 <= move < 9 and board[move] == ' ':

                board[move] = 'X'

                break

            else:

                print("Invalid move! Try again.")

        except ValueError:

            print("Enter a valid number.")
```

```
def play_game():

    """Main game loop."""

    print("Tic-Tac-Toe (Minimax)")

    print_board()

    while True:

        player_move()

        print_board()

        if check_winner('X'):
```

```

        print("You win!")
        break
    if is_full():
        print("It's a draw!")
        break

    computer_move()
    print_board()
    if check_winner('O'):
        print("Computer wins!")
        break
    if is_full():
        print("It's a draw!")
        break

# play_game()

```

AI Termwork 3: Water Jug Problem using DFS

Python

Water Jug Problem using Depth First Search (DFS)

```

capacity_A = 4 # Max capacity of Jug A
capacity_B = 3 # Max capacity of Jug B
goal = 2      # Goal: 2 liters in Jug B
visited = set() # To keep track of states we've already seen to prevent infinite loops

def dfs(state, path):
    """Recursive DFS function to find the goal state."""
    x, y = state

```

```

# If we've already evaluated this state, stop and backtrack
if state in visited:
    return False

# Mark current state as visited and add to our current path
visited.add(state)
path.append(state)

# Goal condition: Jug-B has exactly 2 liters
if y == goal:
    print("Solution Path:")
    for p in path:
        print(p)
    return True

# Define all possible next moves based on production rules
possible_moves = [
    (capacity_A, y), # Rule 1: Fill Jug-A completely
    (x, capacity_B), # Rule 2: Fill Jug-B completely
    (0, y),          # Rule 3: Empty Jug-A
    (x, 0),          # Rule 4: Empty Jug-B
    # Rule 5: Pour Jug-A to Jug-B until B is full or A is empty
    (x - min(x, capacity_B - y), y + min(x, capacity_B - y)),
    # Rule 6: Pour Jug-B to Jug-A until A is full or B is empty
    (x + min(y, capacity_A - x), y - min(y, capacity_A - x))
]

# Try each possible move recursively

```

```

for move in possible_moves:
    if dfs(move, path):
        return True

# If no moves lead to the goal, backtrack by removing the state from path
path.pop()
return False

# Initial State: both jugs are empty (0, 0)
initial_state = (0, 0)
# if not dfs(initial_state, []):
#     print("No solution found.")

```

AI Termwork 4: Water Jug Problem using BFS

Python

Water Jug Problem using Breadth First Search (BFS)

from collections import deque

capacity_A = 4

capacity_B = 3

goal = 2

def bfs():

"""BFS uses a queue to explore all states level by level."""

visited = set()

queue = deque()

Queue stores tuples of (current_state, path_taken_to_reach_it)

queue.append(((0, 0), []))

```
while queue:
```

```
    # Pop the oldest state from the queue (FIFO)
```

```
    (x, y), path = queue.popleft()
```

```
    # Skip if already visited
```

```
    if (x, y) in visited:
```

```
        continue
```

```
    visited.add((x, y))
```

```
    path = path + [(x, y)] # Create a new path list with the current state
```

```
    # Check for goal
```

```
    if y == goal:
```

```
        print("Solution Path:")
```

```
        for p in path:
```

```
            print(p)
```

```
        return
```

```
    # Generate all valid next moves (same as DFS)
```

```
    moves = [
```

```
        (capacity_A, y), # Fill A
```

```
        (x, capacity_B), # Fill B
```

```
        (0, y),          # Empty A
```

```
        (x, 0),          # Empty B
```

```
        (x - min(x, capacity_B - y), y + min(x, capacity_B - y)), # Pour A -> B
```

```
        (x + min(y, capacity_A - x), y - min(y, capacity_A - x)) # Pour B -> A
```

```
    ]
```

```

        # Add unseen moves to the queue
        for move in moves:
            if move not in visited:
                queue.append((move, path))

    print("No solution found.")

# bfs()

```

AI Termwork 5: 8-Puzzle using A* Algorithm

Python

8-Puzzle Problem using A* Algorithm (Manhattan Distance)

```
import heapq
```

```

goal_state = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0] # 0 represents the empty space
]

```

```

def manhattan(state):
    """Calculates the sum of Manhattan distances for all numbered tiles."""
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                # Calculate where the tile SHOULD be
                target_x = (value - 1) // 3

```

```

    target_y = (value - 1) % 3

    # Add the distance from its current pos to its target pos
    distance += abs(i - target_x) + abs(j - target_y)

    return distance

```

```

def get_neighbors(state):
    """Finds all valid board configurations by sliding tiles into the empty space."""
    neighbors = []

    # Find coordinates of the empty space (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    # Directions: Up, Down, Left, Right
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3: # Ensure move is within board limits
            # Create a deep copy of the board
            new_state = [row[:] for row in state]

            # Swap the empty tile (0) with the adjacent tile
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            neighbors.append(new_state)

    return neighbors

```

```

def state_to_tuple(state):
    """Converts a 2D list to a tuple so it can be hashed in a set."""
    return tuple(tuple(row) for row in state)

```

```

def a_star(start):
    """
    A* Search:  $f(n) = g(n) + h(n)$ .
     $g(n)$  = cost to reach node,  $h(n)$  = heuristic (Manhattan distance).
    """
    pq = []
    # Priority Queue stores tuples of: (f_score, g_score, current_state, path)
    heapq.heappush(pq, (manhattan(start), 0, start, []))
    visited = set()

    while pq:
        f, g, state, path = heapq.heappop(pq) # Pop state with lowest f_score

        if state_to_tuple(state) in visited:
            continue
        visited.add(state_to_tuple(state))

        path = path + [state] # Record the step

        if state == goal_state:
            print("Solution Path:")
            for step in path:
                print()
                for row in step:
                    print(row)
            return

        for neighbor in get_neighbors(state):

```

```

        if state_to_tuple(neighbor) not in visited:
            # f = (g + 1) + h(neighbor)
            heapq.heappush(pq, ((g + 1) + manhattan(neighbor), g + 1, neighbor, path))

    print("No solution found.")

start_state = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]
# a_star(start_state)

```

AI Termwork 6: 8-Puzzle using Greedy Best First Search

Python

8-Puzzle using Greedy Best First Search (Manhattan Distance)

import heapq

```

goal_state = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

```

```

def manhattan(state):
    """Calculates Manhattan distance (same as A*)."""
    distance = 0
    for i in range(3):
        for j in range(3):

```

```

    val = state[i][j]
    if val != 0:
        target_x = (val - 1) // 3
        target_y = (val - 1) % 3
        distance += abs(i - target_x) + abs(j - target_y)
    return distance

```

```

def get_neighbors(state):
    """Generates neighbor states (same as A*)."""
    neighbors = []
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j
                directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
                for dx, dy in directions:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < 3 and 0 <= ny < 3:
                        new_state = [row[:] for row in state]
                        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
                        neighbors.append(new_state)
    return neighbors

```

```

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

```

```

def greedy_bfs(start):

```

```

    """

```

Greedy BFS: Only uses heuristic $h(n)$ to determine the next move.

It ignores the path cost $g(n)$, making it faster but not always optimal.

```
"""
```

```
pq = []
```

```
# Priority Queue stores: (h_score, current_state, path)
```

```
heapq.heappush(pq, (manhattan(start), start, []))
```

```
visited = set()
```

```
while pq:
```

```
    h, state, path = heapq.heappop(pq) # Pop state with lowest heuristic
```

```
    if state_to_tuple(state) in visited:
```

```
        continue
```

```
    visited.add(state_to_tuple(state))
```

```
    path = path + [state]
```

```
    if state == goal_state:
```

```
        print("Solution Path:")
```

```
        for step in path:
```

```
            print()
```

```
            for row in step:
```

```
                print(row)
```

```
        return
```

```
    for neighbor in get_neighbors(state):
```

```
        if state_to_tuple(neighbor) not in visited:
```

```
            # Push based ONLY on manhattan distance
```

```
            heapq.heappush(pq, (manhattan(neighbor), neighbor, path))
```

```
print("No solution found.")

start_state = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

# greedy_bfs(start_state)
```

AI Termwork 7: Sudoku Solver using Backtracking (CSP)

Python

Sudoku Solver using Constraint Satisfaction (Backtracking)

```
def print_board(board):
    for row in board:
        print(row)

def find_empty(board):
    """Finds the first empty cell (represented by 0)."""
    for i in range(len(board)):
        for j in range(len(board[0])):
            if board[i][j] == 0:
                return i, j # Return row, col
    return None

def is_valid(board, num, pos):
    """Checks if placing 'num' at 'pos' violates Sudoku rules."""
    row, col = pos
```

1. Row check: Ensure number doesn't exist in the current row

for j in range(len(board[0])):

if board[row][j] == num and col != j:

return False

2. Column check: Ensure number doesn't exist in the current column

for i in range(len(board)):

if board[i][col] == num and row != i:

return False

3. Grid check: Ensure number doesn't exist in the 3x3 subgrid

n = int(len(board) ** 0.5) # For 9x9, n is 3

box_x = col // n

box_y = row // n

for i in range(box_y * n, box_y * n + n):

for j in range(box_x * n, box_x * n + n):

if board[i][j] == num and (i, j) != pos:

return False

return True

def solve(board):

"""Recursive backtracking function to solve the board."""

find = find_empty(board)

if not find:

return True # Board is fully solved!

else:

row, col = find

```

# Try placing numbers 1 through 9
for num in range(1, len(board) + 1):
    if is_valid(board, num, (row, col)):
        board[row][col] = num # Tentatively place the number

        # Recursively try to solve the rest of the board
        if solve(board):
            return True

        # If it leads to a dead end, undo the placement (backtrack)
        board[row][col] = 0

return False # Trigger backtracking to the previous cell

board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

# if solve(board):
#     print("Solved Sudoku:")

```

```
# print_board(board)

# else:

# print("No solution exists.")
```

Machine Learning Termwork 1: Simple Linear Regression (Least Squares)

Python

Simple Linear Regression using Least Squares Method (from scratch)

```
import numpy as np
```

Sample dataset

```
X = np.array([1, 2, 3, 4, 5])
```

```
Y = np.array([2, 4, 5, 4, 5])
```

Calculate mean of X and Y

```
x_mean = np.mean(X)
```

```
y_mean = np.mean(Y)
```

Calculate slope (m) and intercept (b) using Least Squares Formulas

$m = \frac{\text{Sum}((X - X_{\text{mean}}) * (Y - Y_{\text{mean}}))}{\text{Sum}((X - X_{\text{mean}})^2)}$

```
numerator = np.sum((X - x_mean) * (Y - y_mean))
```

```
denominator = np.sum((X - x_mean) ** 2)
```

```
m = numerator / denominator
```

$b = Y_{\text{mean}} - m * X_{\text{mean}}$

```
b = y_mean - m * x_mean
```

```
print("Slope (m):", m)
```

```
print("Intercept (b):", b)
```

```
# Prediction function using  $y = mx + b$ 
def predict(x):
    return m * x + b

# Test prediction
x_test = 6
# print("Prediction for x =", x_test, "is", predict(x_test))
```

Machine Learning Termwork 2: Simple Linear Regression (Gradient Descent)

Python

Simple Linear Regression using Gradient Descent (from scratch)

```
import numpy as np
```

Sample dataset

```
X = np.array([1, 2, 3, 4, 5])
```

```
Y = np.array([2, 4, 5, 4, 5])
```

Initialize parameters

```
m = 0 # slope
```

```
b = 0 # intercept
```

```
learning_rate = 0.01
```

```
epochs = 1000
```

```
n = len(X)
```

Gradient Descent Loop

```
for _ in range(epochs):
```

```
    # Make current predictions
```

```
    Y_pred = m * X + b
```

```

# Calculate gradients (derivatives of Mean Squared Error)
dm = (-2/n) * np.sum(X * (Y - Y_pred))
db = (-2/n) * np.sum(Y - Y_pred)

# Update parameters using gradients and learning rate
m = m - learning_rate * dm
b = b - learning_rate * db

print("Slope (m):", m)
print("Intercept (b):", b)

def predict(x):
    return m * x + b

x_test = 6
# print("Prediction for x =", x_test, "is", predict(x_test))

```

Machine Learning Termwork 3: Multiple Linear Regression (Gradient Descent)

Python

Multiple Linear Regression using Gradient Descent (from scratch)

import numpy as np

Sample dataset (house features: size, bedrooms)

```

X = np.array([
    [1000, 2],
    [1500, 3],
    [1800, 4],
    [2400, 3],
    [3000, 5]
])

```

```
], dtype=float)

Y = np.array([200000, 300000, 350000, 400000, 500000], dtype=float)

# Normalize features (Standardization) to help Gradient Descent converge faster
X = (X - X.mean(axis=0)) / X.std(axis=0)

n_samples, n_features = X.shape

# Initialize weights (array) and bias (scalar)
W = np.zeros(n_features)
b = 0
learning_rate = 0.01
epochs = 1000

# Gradient Descent Loop
for _ in range(epochs):
    # Predict using Dot Product:  $Y_{pred} = (X \cdot W) + b$ 
    Y_pred = np.dot(X, W) + b

    # Calculate Gradients
    dW = (-2/n_samples) * np.dot(X.T, (Y - Y_pred))
    db = (-2/n_samples) * np.sum(Y - Y_pred)

    # Update Weights and Bias
    W -= learning_rate * dW
    b -= learning_rate * db

print("Weights:", W)
print("Bias:", b)
```

```
def predict(x):
    # Make sure to normalize test data using the training data's mean/std
    x = (x - X.mean(axis=0)) / X.std(axis=0)
    return np.dot(x, W) + b

# Test prediction
x_test = np.array([2000, 3])
# print("Prediction:", predict(x_test))
```

Machine Learning Termwork 4: Decision Tree (ID3)

Python

Decision Tree using ID3 Algorithm (Play Tennis Dataset)

import math

import pandas as pd

Dataset setup

```
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',
               'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Mild', 'Hot', 'Cool', 'Mild', 'Mild',
                   'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',
                'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak',
            'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes',
                  'No']
}

df = pd.DataFrame(data)
```

```

def entropy(target):
    """Calculates the entropy of a target column to measure impurity."""
    values = target.value_counts()
    total = len(target)
    ent = 0
    for v in values:
        p = v / total
        ent -= p * math.log2(p)
    return ent

def information_gain(df, feature, target="PlayTennis"):
    """Calculates how much a feature reduces entropy (Higher is better)."""
    total_entropy = entropy(df[target]) # Entropy of dataset before split
    values = df[feature].unique()
    weighted_entropy = 0

    # Calculate weighted average entropy after splitting by the feature
    for v in values:
        subset = df[df[feature] == v]
        weighted_entropy += (len(subset)/len(df)) * entropy(subset[target])

    # Info Gain = Entropy(before) - Weighted_Entropy(after)
    return total_entropy - weighted_entropy

def id3(df, features, target="PlayTennis"):
    """Recursive ID3 algorithm to build the Decision Tree."""
    # Base cases:
    # 1. If all target values are the same, return that class

```

```

if len(df[target].unique()) == 1:
    return df[target].iloc[0]

# 2. If no more features to split on, return majority class
if not features:
    return df[target].mode()[0]

# Find the feature with the highest Information Gain
gains = [information_gain(df, f, target) for f in features]
best_feature = features[gains.index(max(gains))]

# Initialize the tree node
tree = {best_feature: {}}

# Create branches for each value of the best feature
for value in df[best_feature].unique():
    subset = df[df[best_feature] == value]
    # Recursively build subtree excluding the feature we just split on
    subtree = id3(subset, [f for f in features if f != best_feature], target)
    tree[best_feature][value] = subtree

return tree

features = list(df.columns[:-1]) # All columns except the target
# tree = id3(df, features)
# print("Decision Tree:")
# print(tree)

```

Machine Learning Termwork 5: Naive Bayes Algorithm

Python

```
# Naive Bayes Algorithm from scratch (Play Tennis Dataset)
```

```
import pandas as pd
```

```
import numpy as np
```

```
# Dataset setup
```

```
data = {
```

```
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',  
               'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
```

```
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Mild', 'Hot', 'Cool', 'Mild', 'Mild',  
                   'Mild', 'Hot', 'Mild'],
```

```
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',  
                'Normal', 'Normal', 'High', 'Normal', 'High'],
```

```
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak',  
            'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
```

```
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes',  
                  'No']
```

```
}
```

```
df = pd.DataFrame(data)
```

```
def train_naive_bayes(df, target):
```

```
    """Calculates prior probabilities and likelihoods for the dataset."""
```

```
    classes = df[target].unique()
```

```
    priors = {}
```

```
    likelihood = {}
```

```
    for c in classes:
```

```
        df_c = df[df[target] == c]
```

```
        # Prior  $P(\text{Class}) = (\text{Count of Class}) / (\text{Total Rows})$ 
```

```
        priors[c] = len(df_c) / len(df)
```

```
        likelihood[c] = {}
```

```

# Calculate likelihood for each feature value given the class
for col in df.columns:
    if col != target:
        likelihood[c][col] = {}
        for val in df[col].unique():
            # Count how many times this value appears in this class
            count = len(df_c[df_c[col] == val])
            # Laplace smoothing formula: (count + 1) / (class_count + unique_values)
            # Prevents 0 probabilities
            likelihood[c][col][val] = (count + 1) / (len(df_c) + len(df[col].unique()))

return priors, likelihood

def predict(sample, priors, likelihood):
    """Predicts the class for a new sample using Naive Bayes."""
    posteriors = {}

    # Calculate posterior probability for each class (using log to prevent underflow)
    for c in priors:
        prob = np.log(priors[c])
        for feature in sample:
            val = sample[feature]
            # Add log likelihood (multiplication becomes addition with logs)
            # get(val, 1e-6) provides a tiny fallback value if 'val' was never seen
            prob += np.log(likelihood[c][feature].get(val, 1e-6))
        posteriors[c] = prob

    # Return the class with the highest probability

```

```

return max(posterioriors, key=posterioriors.get)

# priors, likelihood = train_naive_bayes(df, 'PlayTennis')
# sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
# result = predict(sample, priors, likelihood)
# print("Prediction:", result)

```

Machine Learning Termwork 6: K-Nearest Neighbors (KNN)

Python

K-Nearest Neighbors (KNN) from scratch

import numpy as np

Sample dataset (features + labels)

```

X = np.array([
    [1, 2], [2, 3], [3, 3], [6, 5], [7, 7], [8, 6]
])

```

y = np.array([0, 0, 0, 1, 1, 1]) # labels

def euclidean_distance(a, b):

"""Standard distance formula: $\sqrt{\sum (a - b)^2}$."""

return np.sqrt(np.sum((a - b) ** 2))

def knn(X, y, query, k=3):

"""Finds the 'k' nearest training points and votes on the majority class."""

distances = []

1. Calculate distance from query point to all training data

for i in range(len(X)):

dist = euclidean_distance(query, X[i])

```

distances.append((dist, y[i])) # Store tuple of (distance, label)

# 2. Sort list by distance (ascending)
distances.sort(key=lambda x: x[0])

# 3. Take first k neighbors
neighbors = distances[:k]

# 4. Perform Majority voting
labels = [label for _, label in neighbors]
prediction = max(set(labels), key=labels.count) # Find most frequent label

return prediction

query_point = np.array([5, 5])
# result = knn(X, y, query_point, k=3)
# print("Prediction:", result)

```

Machine Learning Termwork 7: K-Means Clustering

Python

K-Means Clustering from scratch

import numpy as np

Sample dataset

```

X = np.array([
    [1, 2], [1.5, 1.8], [5, 8], [8, 8], [1, 0.6], [9, 11]
])

```

k = 2 # Number of clusters we want to form

```

# Initialize centroids randomly by picking 'k' random points from dataset
np.random.seed(0)

# choice(len, k, replace=False) selects k unique indices
centroids = X[np.random.choice(len(X), k, replace=False)]

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def kmeans(X, k, centroids, max_iters=100):
    for _ in range(max_iters):
        # Initialize empty lists for each cluster
        clusters = [[] for _ in range(k)]

        # Step 1: Assign points to nearest centroid
        for point in X:
            # Measure distance from point to all centroids
            distances = [euclidean_distance(point, c) for c in centroids]

            # Find index of the closest centroid
            cluster_index = np.argmin(distances)

            clusters[cluster_index].append(point)

        new_centroids = []

        # Step 2: Compute new centroids (mean of assigned points)
        for cluster in clusters:
            if cluster:
                # Calculate mean along columns (axis=0) to find center
                new_centroids.append(np.mean(cluster, axis=0))
            else:
                # If a cluster ends up empty, keep the old centroid to prevent errors

```

```

        new_centroids.append(centroids[len(new_centroids)])

new_centroids = np.array(new_centroids)

# Step 3: Stop early if centroids don't change (algorithm converged)
if np.allclose(centroids, new_centroids):
    centroids = new_centroids
    break

centroids = new_centroids

return clusters, centroids

# clusters, centroids = kmeans(X, k, centroids)
# print("Clusters:")
# for i, cluster in enumerate(clusters):
#     print(f"Cluster

```